CO4 ASSESSMENT TOOL 1

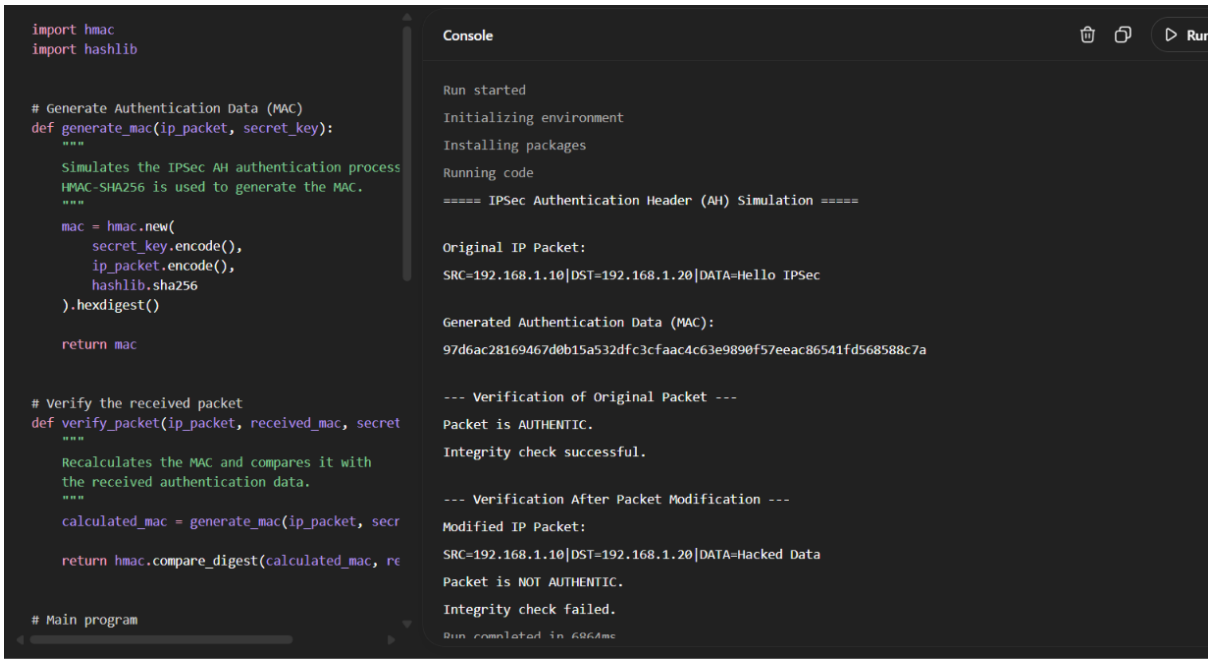
NAME:S.Dhivyasri

REG NO.:192511246

COURSE :Cryptography and network security

COURSE CODE :CSA5113

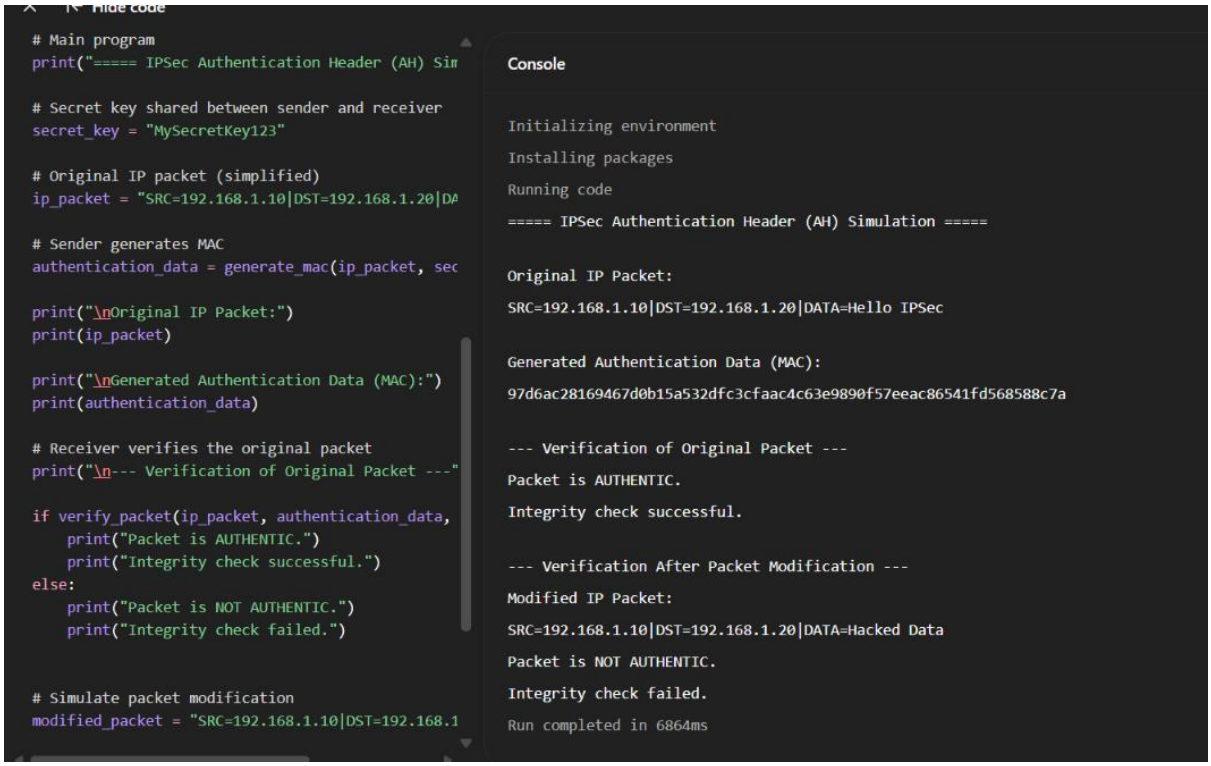
1.Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.



```
import hmac
import hashlib


# Generate Authentication Data (MAC)
def generate_mac(ip_packet, secret_key):
    """
    Simulates the IPSec AH authentication process
    HMAC-SHA256 is used to generate the MAC.
    """
    mac = hmac.new(
        secret_key.encode(),
        ip_packet.encode(),
        hashlib.sha256
    ).hexdigest()

    return mac


# Verify the received packet
def verify_packet(ip_packet, received_mac, secret
    """
    Recalculates the MAC and compares it with
    the received authentication data.
    """
    calculated_mac = generate_mac(ip_packet, secr

    return hmac.compare_digest(calculated_mac, re


# Main program
```

Console

```
Run started
Initializing environment
Installing packages
Running code
===== IPSec Authentication Header (AH) Simulation =====

Original IP Packet:
SRC=192.168.1.10|DST=192.168.1.20|DATA=Hello IPSec

Generated Authentication Data (MAC):
97d6ac28169467d0b15a532dfc3cfaac4c63e9890f57eeac86541fd568588c7a

--- Verification of Original Packet ---
Packet is AUTHENTIC.
Integrity check successful.

--- Verification After Packet Modification ---
Modified IP Packet:
SRC=192.168.1.10|DST=192.168.1.20|DATA=Hacked Data
Packet is NOT AUTHENTIC.
Integrity check failed.
Run completed in 6864ms
```



```
# Main program
print("===== IPSec Authentication Header (AH) Sim

# Secret key shared between sender and receiver
secret_key = "MySecretKey123"

# Original IP packet (simplified)
ip_packet = "SRC=192.168.1.10|DST=192.168.1.20|DA

# Sender generates MAC
authentication_data = generate_mac(ip_packet, sec

print("\nOriginal IP Packet:")
print(ip_packet)

print("\nGenerated Authentication Data (MAC):")
print(authentication_data)

# Receiver verifies the original packet
print("\n--- Verification of Original Packet ---"

if verify_packet(ip_packet, authentication_data,
    print("Packet is AUTHENTIC.")
    print("Integrity check successful.")
else:
    print("Packet is NOT AUTHENTIC.")
    print("Integrity check failed.")


# Simulate packet modification
modified_packet = "SRC=192.168.1.10|DST=192.168.1
```

Console

```
Initializing environment
Installing packages
Running code
===== IPSec Authentication Header (AH) Simulation =====

Original IP Packet:
SRC=192.168.1.10|DST=192.168.1.20|DATA=Hello IPSec

Generated Authentication Data (MAC):
97d6ac28169467d0b15a532dfc3cfaac4c63e9890f57eeac86541fd568588c7a

--- Verification of Original Packet ---
Packet is AUTHENTIC.
Integrity check successful.

--- Verification After Packet Modification ---
Modified IP Packet:
SRC=192.168.1.10|DST=192.168.1.20|DATA=Hacked Data
Packet is NOT AUTHENTIC.
Integrity check failed.
Run completed in 6864ms
```

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

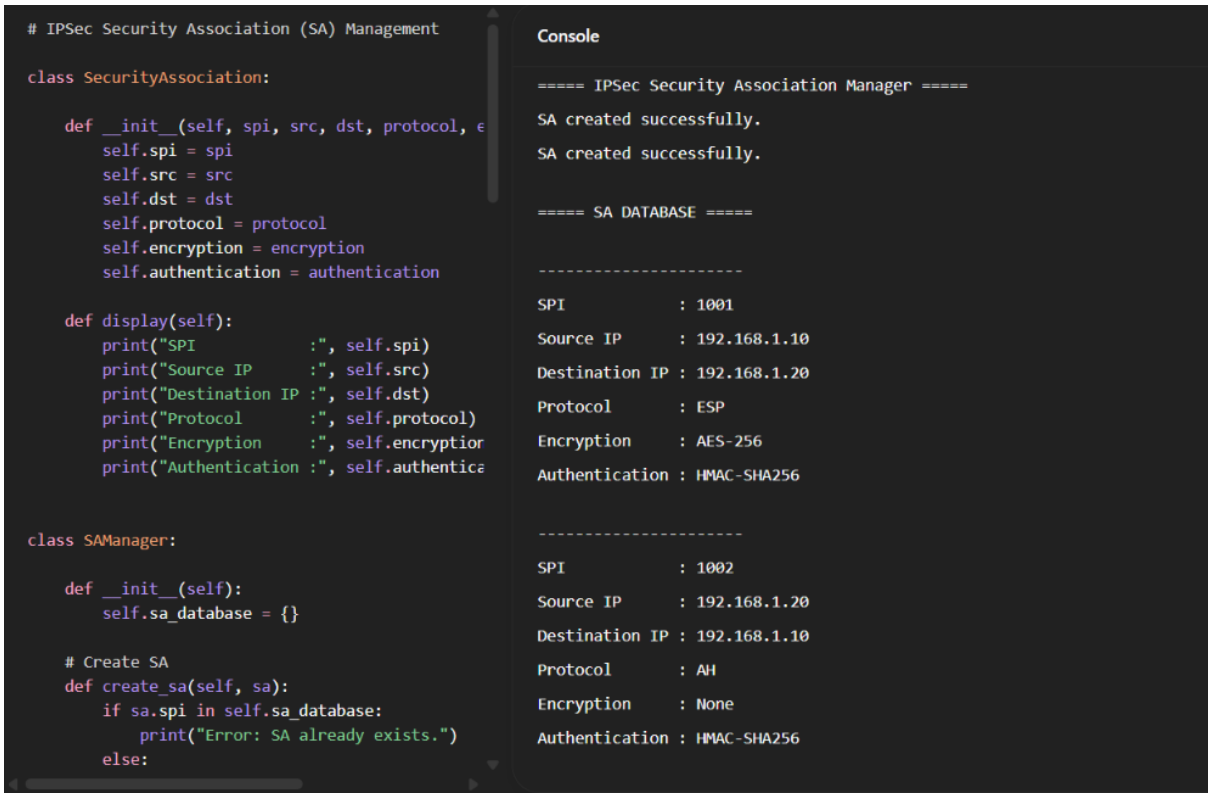
```
from cryptography.hazmat.primitives.ciphers impor
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives import paddir
import os


# AES encryption
def encrypt_payload(payload, key, iv):
    padder = padding.PKCS7(128).padder()
    padded_data = padder.update(payload.encode())

    cipher = Cipher(algorithms.AES(key), modes.CE
    encryptor = cipher.encryptor()

    ciphertext = encryptor.update(padded_data) +
    return ciphertext


# AES decryption
def decrypt_payload(ciphertext, key, iv):
    cipher = Cipher(algorithms.AES(key), modes.CE
    decryptor = cipher.decryptor()

    padded_data = decryptor.update(ciphertext) +

    unpadder = padding.PKCS7(128).unpadder()
    plaintext = unpadder.update(padded_data) + ur

    return plaintext.decode()
```

```
Console                                          Run

Run started
Initializing environment
Installing packages
Running code
===== IPSec ESP Simulation =====

Original Payload:
Hello, this is a confidential IP packet.

Encrypted Payload (Hex):
9a9204f9dbdf1a75f663fb2afa8ffa47dc6ca2b852f2fa79f42b51ecae550339fa1a828b9504b578a0f3fefe2e66c514

Authentication Tag (HMAC):
6158f9f56e3f736e68b5c15989299ea8e5dae5e8ec759f59cafd5086893fb4fe

--- ESP Decryption ---
Authentication successful.
Integrity check successful.

Decrypted Payload:
Hello, this is a confidential IP packet.
```

```
    return h.finalize()


# Main program
print("===== IPSec ESP Simulation =====")

# AES-256 encryption key
encryption_key = b"0123456789abcdef0123456789abcd

# Authentication key
authentication_key = b"AuthenticationKey12345678

# Initialization Vector
iv = os.urandom(16)

# Original IP packet payload
payload = "Hello, this is a confidential IP packe

print("\nOriginal Payload:")
print(payload)

# ---------------- ESP ENCRYPTION ----------------

ciphertext = encrypt_payload(payload, encryption_

# Generate authentication tag
auth_data = iv + ciphertext
mac = generate_hmac(auth_data, authentication_key

print("\nEncrypted Payload (Hex):")
print(ciphertext.hex())
```

```
Console                                          Run

===== IPSec ESP Simulation =====

Original Payload:
Hello, this is a confidential IP packet.

Encrypted Payload (Hex):
9a9204f9dbdf1a75f663fb2afa8ffa47dc6ca2b852f2fa79f42b51ecae550339fa1a828b9504b578a0f3fefe2e66c514

Authentication Tag (HMAC):
6158f9f56e3f736e68b5c15989299ea8e5dae5e8ec759f59cafd5086893fb4fe

--- ESP Decryption ---
Authentication successful.
Integrity check successful.

Decrypted Payload:
Hello, this is a confidential IP packet.

--- Tampering Test ---
Authentication failed!
Tampered packet detected.
Run completed in 3184ms
```

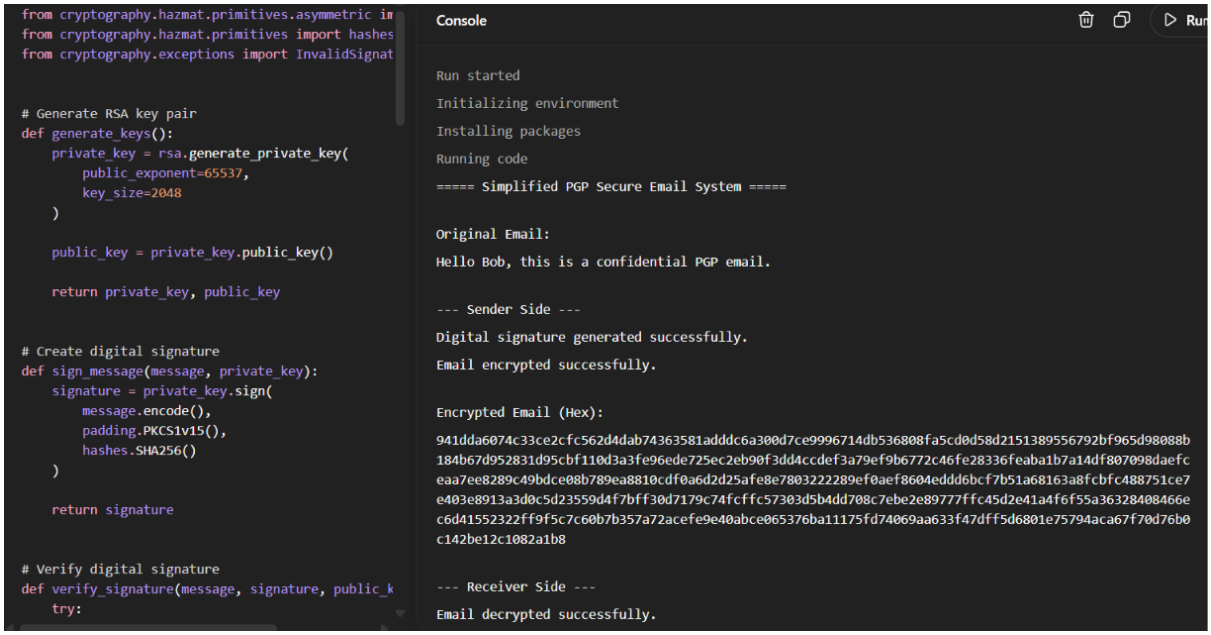
3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.



```
# IPSec Security Association (SA) Management

class SecurityAssociation:

    def __init__(self, spi, src, dst, protocol, e
        self.spi = spi
        self.src = src
        self.dst = dst
        self.protocol = protocol
        self.encryption = encryption
        self.authentication = authentication

    def display(self):
        print("SPI            :", self.spi)
        print("Source IP      :", self.src)
        print("Destination IP :", self.dst)
        print("Protocol       :", self.protocol)
        print("Encryption     :", self.encryptior
        print("Authentication :", self.authentica


class SAManager:

    def __init__(self):
        self.sa_database = {}

    # Create SA
    def create_sa(self, sa):
        if sa.spi in self.sa_database:
            print("Error: SA already exists.")
        else:
```

```
Console

===== IPSec Security Association Manager =====
SA created successfully.
SA created successfully.

===== SA DATABASE =====

----------------------
SPI            : 1001
Source IP      : 192.168.1.10
Destination IP : 192.168.1.20
Protocol       : ESP
Encryption     : AES-256
Authentication : HMAC-SHA256

----------------------
SPI            : 1002
Source IP      : 192.168.1.20
Destination IP : 192.168.1.10
Protocol       : AH
Encryption     : None
Authentication : HMAC-SHA256
```

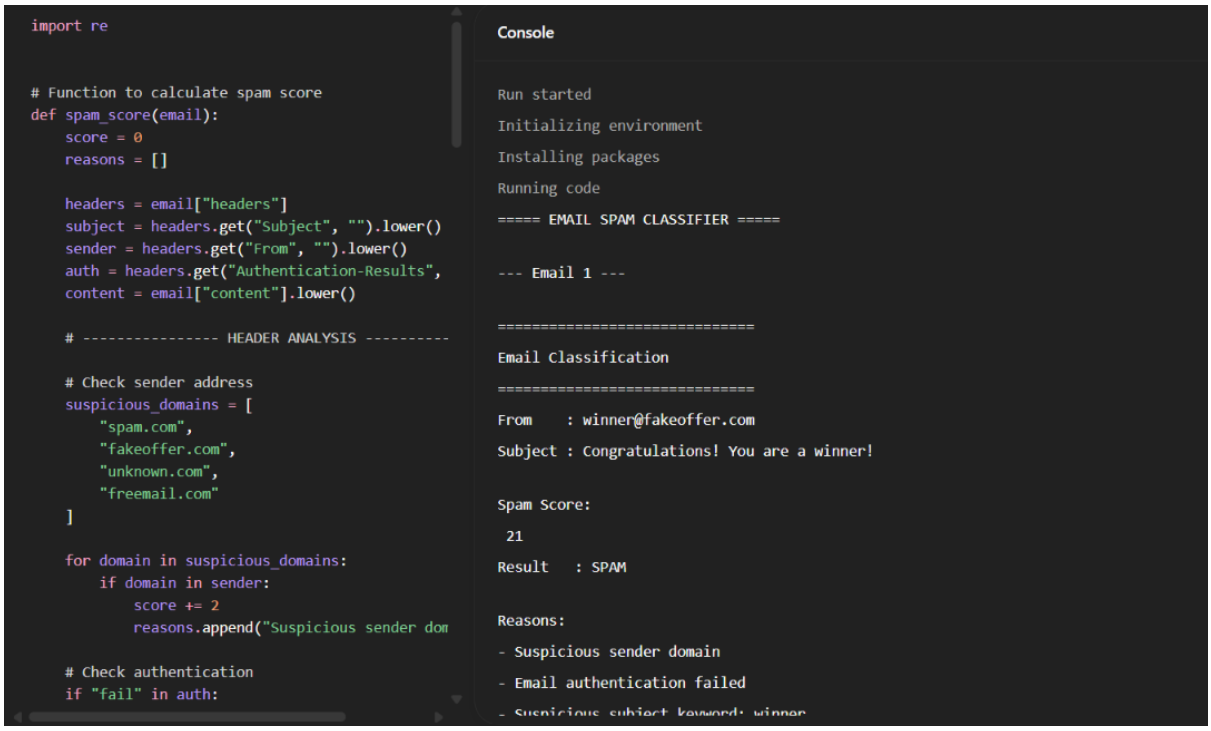
4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.



```
from cryptography.hazmat.primitives.asymmetric im
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignat


# Generate RSA key pair
def generate_keys():
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    public_key = private_key.public_key()

    return private_key, public_key


# Create digital signature
def sign_message(message, private_key):
    signature = private_key.sign(
        message.encode(),
        padding.PKCS1v15(),
        hashes.SHA256()
    )

    return signature


# Verify digital signature
def verify_signature(message, signature, public_k
    try:
```

```
Console

Run started
Initializing environment
Installing packages
Running code
===== Simplified PGP Secure Email System =====

Original Email:
Hello Bob, this is a confidential PGP email.

--- Sender Side ---
Digital signature generated successfully.
Email encrypted successfully.

Encrypted Email (Hex):
941dda6074c33ce2cfc562d4dab74363581adddc6a300d7ce9996714db536808fa5cd0d58d2151389556792bf965d98088b
184b67d952831d95cbf110d3a3fe96ede725ec2eb90f3dd4ccdef3a79ef9b6772c46fe28336feaba1b7a14df807098daefc
eaa7ee8289c49bdce08b789ea8810cdf0a6d2d25afe8e7803222289ef0aef8604eddd6bcf7b51a68163a8fcbfc488751ce7
e403e8913a3d0c5d23559d4f7bff30d7179c74fcffc57303d5b4dd708c7ebe2e89777ffc45d2e41a4f6f55a36328408466e
c6d41552322ff9f5c7c60b7b357a72acefe9e40abce065376ba11175fd74069aa633f47dff5d6801e75794aca67f70d76b0
c142be12c1082a1b8

--- Receiver Side ---
Email decrypted successfully.
```

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.



```
import re


# Function to calculate spam score
def spam_score(email):
    score = 0
    reasons = []

    headers = email["headers"]
    subject = headers.get("Subject", "").lower()
    sender = headers.get("From", "").lower()
    auth = headers.get("Authentication-Results",
    content = email["content"].lower()

    # --------------- HEADER ANALYSIS ----------

    # Check sender address
    suspicious_domains = [
        "spam.com",
        "fakeoffer.com",
        "unknown.com",
        "freemail.com"
    ]

    for domain in suspicious_domains:
        if domain in sender:
            score += 2
            reasons.append("Suspicious sender dom

    # Check authentication
    if "fail" in auth:
```

```
Console

Run started
Initializing environment
Installing packages
Running code
===== EMAIL SPAM CLASSIFIER =====

--- Email 1 ---

============================
Email Classification
============================
From    : winner@fakeoffer.com
Subject : Congratulations! You are a winner!

Spam Score:
 21
Result   : SPAM

Reasons:
- Suspicious sender domain
- Email authentication failed
- Suspicious subject keyword: winner
```